Exam 1 Review Solutions R

CS 184: Foundations of Computer Graphics

1 Sampling and Antialiasing

1. **True or false: sampling and supersampling.** Mark each statement **T** or **F**, and briefly justify your answer.

   Spring 2024, Exam 1, Question 1(a)–(b)

   (a) We get no aliasing if we sample at more than twice the highest frequency in the signal.**T / F**

   (b) In supersampled rasterization of N triangles, it is best to supersample and resolve each triangle into the framebuffer, one by one. **T / F**

   Solution: (a) True. For a band-limited signal, sampling strictly above twice its highest frequency avoids aliasing (Nyquist criterion).
   (b) False. Rasterize all triangles into the supersample buffer first, and resolve to the framebuffer only after the full scene has been accumulated. Resolving after each individual triangle discards needed subpixel information.

2. **True or false: filtering and aliasing artifacts.** Mark **T** or **F** for each statement.

   Summer 2025, Exam 1, Question 1(h)–(i)

   (a) A Gaussian convolution kernel is a form of low-pass filtering over the frequency domain. **T / F**

   (b) Jaggies, the wagon-wheel effect, and Moiré patterns are all artifacts of undersampling.**T / F**

   Solution: (a) True. Gaussian smoothing suppresses high-frequency components (low-pass filtering).
   (b) True. Spatial jaggies, temporal wagon-wheel motion, and Moiré patterns all arise from inadequate sampling.

3. **Temporal antialiasing.** A prototype video camera produces temporal aliasing (e.g., the wagon-wheel effect) when recording fast-moving details. It records at 60 frames per second; its shutter is open for 0.00025 seconds in each frame. **Which options might reduce temporal aliasing? Circle all that apply.**

Spring 2021, Exam 1, Question 3(c)

(a) ((i) points) Increase the time the shutter is open for each video frame to 0.01 seconds.

(b) ((ii) points) Blur across time by replacing every video frame with the average of the five video frames centered at that time.

(c) ((iii) points) Increase the frame rate to produce a video at 240 frames per second.

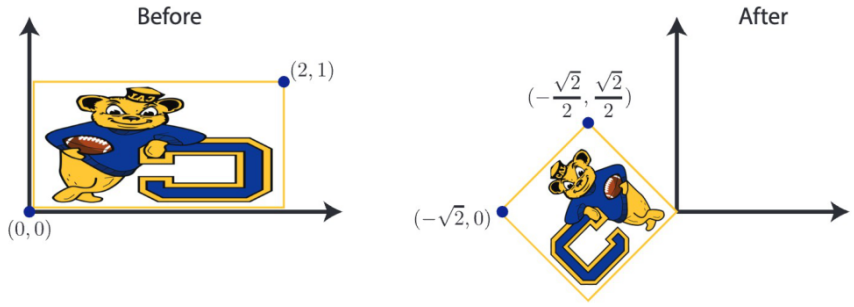
(d) ((iv) points) Increase the frame rate to 240 frames per second and average every four frames to produce a 60-frame-per-second video.

Solution: Circle (i), (iii), and (iv). (i) A longer exposure adds pre-sampling motion blur; (iii) a higher frame rate raises the Nyquist frequency; (iv) recording at 240 fps and averaging groups of four is temporal supersampling with a low-pass resolve. Option (ii) averages *already-aliased* 60-fps images and does not undo that aliasing.

2 Transforms and Coordinate Systems

1. **Composing transformations.** Consider the *before* and *after* images below. Use 2D homogeneous coordinates and column vectors.

Spring 2022, Exam 1, Question 3(a)(i)–(iii)



(a) Write down a sequence of basic transformations (translation, scaling, and/or rotation) which maps the left image to the right image.

Solution: Scale x by $-\frac{1}{2}$ while leaving y unchanged; then rotate counterclockwise by $45°$ around the origin. There is no translation.

$$\mathbf{p}' = R(45°)\, S(-\tfrac{1}{2}, 1)\, \mathbf{p}.$$

(b) Determine the entries of the corresponding 3×3 matrix by multiplying the matrices for your basic transformations. Show the order of multiplication.

Solution: Since the rightmost operation applies first,

$$M = \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & 0 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -\frac{1}{2} & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \boxed{\begin{bmatrix} -\frac{\sqrt{2}}{4} & -\frac{\sqrt{2}}{2} & 0 \\ -\frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix}}.$$

(c) Derive the matrix a second way using the *coordinate-system transform*: identify the transformed origin and basis vectors and use them as the columns of the matrix. Briefly explain.

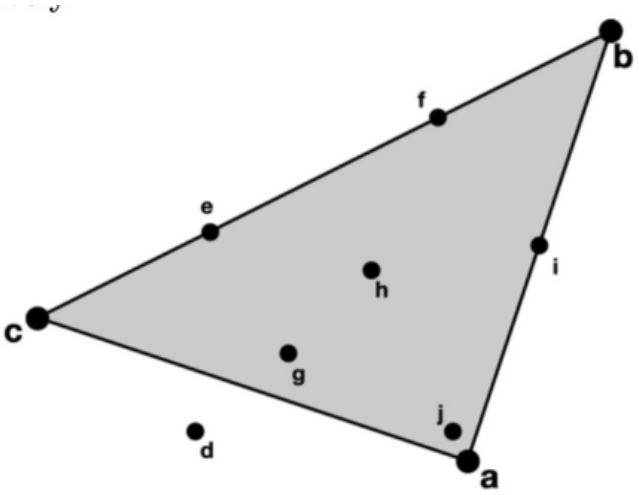
Solution: The transformed origin is $o = (0, 0)$; the two transformed basis vectors are $u = M(1, 0) = (-\frac{\sqrt{2}}{4}, -\frac{\sqrt{2}}{4})$ and $v = M(0, 1) = (-\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2})$. Placing u, v, o in the columns of the homogeneous coordinate-system matrix reproduces the matrix in part (b).

3 Barycentric Coordinates and Texture Mapping

1. **Barycentric coordinates.** Each row lists barycentric coordinates (α, β, γ) relative to vertices a, b, c, respectively. Match each row to a labeled point in the figure. The labels a, b, c denote triangle vertices, not list entries.

Spring 2023, Exam 1, Question 5(a)

#	α	β	γ	Point
1	0.3333	0.3333	0.333	_____
2	0.9	0.05	0.05	_____
3	0	0.3	0.7	_____
4	0	0.7	0.3	_____
5	0.5	0.5	0	_____
6	0.45	0.1	0.45	_____
7	0	0	1	_____
8	0.55	−0.1	0.55	_____



Solution:

Row	1	2	3	4	5	6	7	8
Point	**h**	**j**	**e**	**f**	**i**	**g**	**c**	**d**

All barycentric weights sum to 1. A zero weight places the point on the opposite edge; a negative weight indicates a point outside the triangle.

2. **True or false: mipmaps.** Assuming that the (u, v) coordinates of the vertices stay the same, as a textured triangle on the display becomes smaller, the mipmap level used in texture mapping the triangle increases. **T / F**

Spring 2022, Exam 1, Question 2(d)

Solution: True. With the same (u, v) coordinates mapped onto fewer display pixels, each screen pixel covers a larger texture footprint. The chosen mip level increases, selecting a lower-resolution prefiltered texture.

3. **Choose a mipmap level.** A photo-browser stores a 12 MP image (4000×3000 pixels) as a mipmapped texture, then displays it on a texture-mapped rectangle. The original prompt states a width-to-height ratio of $4/3$. The renderer uses bilinear filtering on the *nearest single* mipmap level; level 0 is the original image.

Spring 2021, Exam 1, Question 3(d)(i)

If the displayed rectangle covers 1800×1200 screen pixels, which mipmap level will be used? Show how you determine the texture-to-screen sampling footprint.

Solution: Texture footprint per display pixel: $|\partial u/\partial x| = 4000/1800 \approx 2.22$, $|\partial v/\partial y| = 3000/1200 = 2.5$, and the cross derivatives vanish. Thus

$$D = \log_2\big(\max(2.22, 2.5)\big) \approx 1.32.$$

The nearest integer level is $\boxed{1}$ (zero-indexed).

4 Rasterization and the Graphics Pipeline

1. **Write a filled-circle rasterizer.** The origin is at the upper-left of the framebuffer, each pixel covers one unit square, and pixel (i, j) has center $(i + \frac{1}{2}, j + \frac{1}{2})$. Use the structures below.

Spring 2024, Exam 1, Question 2(a)(i)–(ii)

```
struct Point2D { float x, y; };
struct Color   { float R, G, B; };
struct Circle  { Point2D center; float radius; Color color; };
struct Framebuffer {
    vector<Color> pixels;   // width * height elements, preallocated
    int width, height;
};
```

(a) Write a helper which tests whether point `p` is inside circle `c`.

```
bool InsideCircle(const Point2D &p, const Circle &c) {
    // Your code here
}
```

Solution:

```
bool InsideCircle(const Point2D &p, const Circle &c) {
    float dx = p.x - c.center.x;
    float dy = p.y - c.center.y;
    return dx*dx + dy*dy < c.radius*c.radius;
}
```

The strict inequality excludes points exactly on the circle boundary.

(b) Write `DrawFilledCircle`, using your helper. Overwrite covered framebuffer pixels (no need to clear the framebuffer). Sample fewer than *all* framebuffer pixels, and ensure that you never access memory outside the framebuffer.

```
void DrawFilledCircle(const Circle &c, Framebuffer &fb) {
    // Your code here
}
```

Solution:

```
void DrawFilledCircle(const Circle &c, Framebuffer &fb) {
    int i0 = max(0, (int)floor(c.center.x - c.radius));
    int i1 = min(fb.width, (int)ceil(c.center.x + c.radius));
    int j0 = max(0, (int)floor(c.center.y - c.radius));
    int j1 = min(fb.height, (int)ceil(c.center.y + c.radius));
    for (int j = j0; j < j1; ++j) {
        for (int i = i0; i < i1; ++i) {
            Point2D p{ i + 0.5f, j + 0.5f };
            if (InsideCircle(p, c))
                fb.pixels[j * fb.width + i] = c.color;
        }
    }
}
```

Only pixels in the circle's bounding rectangle are tested, and the bounds are clipped to the framebuffer.

5 Blinn–Phong Reflection

1. **True or false: Lambert's cosine law.** In the Blinn–Phong shading model, the term $\mathbf{l} \cdot \mathbf{n}$ represents the shading effect due to Lambert's cosine law. **T / F**

Spring 2022, Exam 1, Question 2(f)

Solution: True. The diffuse component depends on $\max(0, \mathbf{n} \cdot \mathbf{l})$: Lambert's cosine law reduces illumination as the surface tilts away from the incoming light.

6 Splines, Bézier Curves, and Continuity

1. **Cubic Hermite interpolation.** Assume a curve segment on $0 \le t \le 1$ is constrained by $P(0) = 5$, $P'(0) = 3$, $P(1) = -2$, and $P'(1) = 3$. Derive the cubic polynomial $f(t)$ interpolating those endpoint positions and tangents. Show your work.

   Summer 2025, Exam 1, Question 3(b)

   Solution: Let $f(t) = at^3 + bt^2 + ct + d$, so $f'(t) = 3at^2 + 2bt + c$. The conditions yield

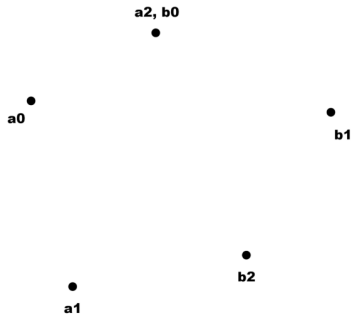
   $$d = 5, \quad c = 3, \quad a + b + 3 + 5 = -2, \quad 3a + 2b + 3 = 3.$$

   Therefore $a + b = -10$, $3a + 2b = 0$, and hence $a = 20$, $b = -30$. Thus

   $$\boxed{f(t) = 20t^3 - 30t^2 + 3t + 5}.$$

2. **de Casteljau for two quadratic Bézier curves.** The figure shows control points for two quadratic Bézier curves with control points (a_0, a_1, a_2) and (b_0, b_1, b_2); their shared endpoint is $a_2 = b_0$.

   Spring 2020, Exam 1, Question 4(a)

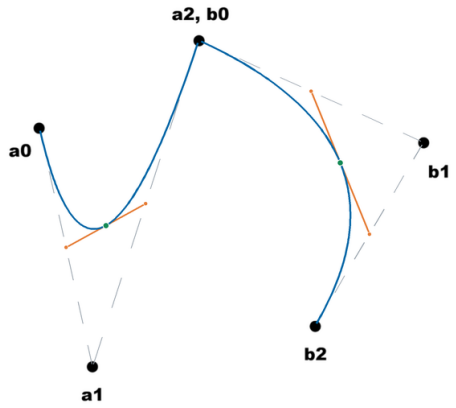
   

   (a) Using de Casteljau's algorithm at $t = \frac{1}{2}$, draw all intermediate control points and identify the midpoint on *each* curve. You need not label intermediate points.

   (b) Sketch the complete two-piece curve on the figure.

   (c) At the joint, is the spline C^0, C^1, or G^1 continuous? Explain briefly using the neighboring control points.

Solution: At $t = \frac{1}{2}$, first compute $A_{01} = (a_0 + a_1)/2$ and $A_{12} = (a_1 + a_2)/2$, then $m_A = (A_{01} + A_{12})/2$. Similarly, $B_{01} = (b_0 + b_1)/2$, $B_{12} = (b_1 + b_2)/2$, and $m_B = (B_{01} + B_{12})/2$.

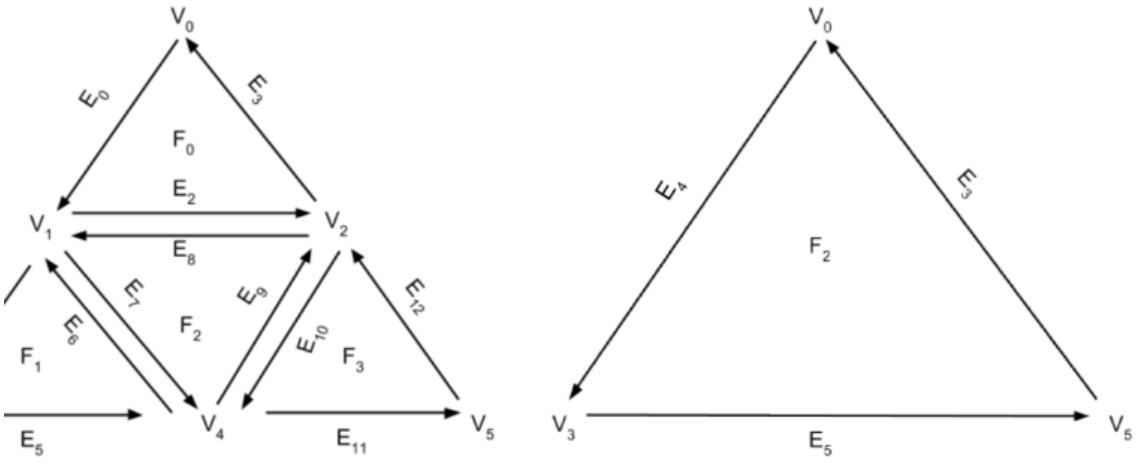


The **blue** paths are the two curves; the orange segment joins first-stage midpoints, and the green dot is the parametric midpoint of each curve. The joint is $\boxed{C^0}$ continuous because $a_2 = b_0$, but it is neither C^1 nor G^1: the one-sided tangents $2(a_2 - a_1)$ and $2(b_1 - b_0)$ are not aligned.

7 Half-Edge Meshes and Local Operations

1. **Half-edge pointer traversal.** Two half-edge meshes are shown below (initial and final). The operations `halfedge()`, `vertex()`, `face()`, `twin()`, and `next()` can be used both to read and to assign links. For example, `E2->twin() = V1->halfedge()` assigns a pointer.

Spring 2020, Exam 1, Question 4(c)–(d)



(a) Link is given only a pointer to halfedge E_4. He wants the initial mesh to have the connectivity of the final mesh. Assign E_4's new face using the *fewest calls*; assume he cannot access unshown elements. Write one line.

Solution:

```
E4->face() = E4->next()->next()->twin()->face();
```

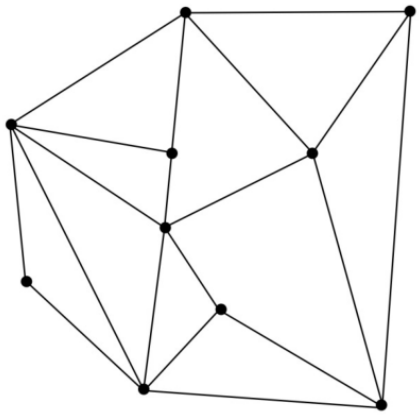
(b) Considering the same change, what is the minimum number of elements whose `next()` links must be reassigned to *guarantee* the final mesh? Assume nothing outside the shown region changes.

Solution: $\boxed{2}$ elements: E_3 and E_5 must have their `next()` links reassigned.

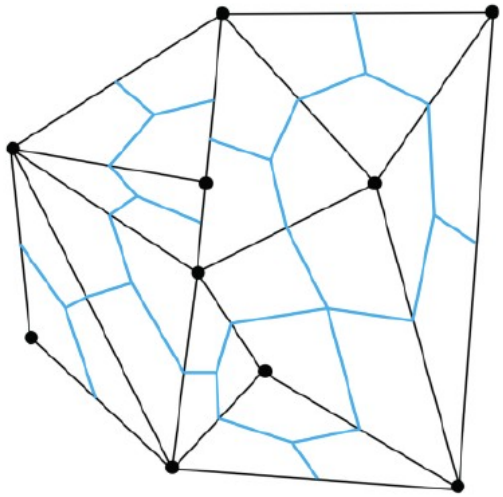
8 Mesh Subdivision

1. **One Catmull–Clark subdivision step.** Draw *one* level of Catmull–Clark subdivision of the mesh below. Indicate the new vertices and edges clearly; use the empty space to redraw the result if needed.

Spring 2023, Exam 1, Question 5(c)(ii)



Solution: A subdivided topology (new edges shown in blue):



Each original face gains a face point, each edge gains an edge point, and each old face is split into quads linking its face point to adjacent edge points and updated old vertices.

2. **Subdivision counts.** Fill in the blanks.

Spring 2017, Exam 1, Question 3(a)–(b)

(a) A triangle mesh with N triangles will contain _____ triangles after $K > 0$ applications of Loop subdivision.

(b) A general mesh starts with V extraordinary vertices and P non-quad polygons. After K applications of Catmull–Clark subdivision, it will have _____ extraordinary vertices.

Solution: **(a)** $\boxed{N \cdot 4^K}$ triangles (each triangle becomes four at each level).
(b) $\boxed{V + P}$ extraordinary vertices after $K > 0$ Catmull–Clark steps; each original nonquad contributes one additional extraordinary face point after its first split.

9 Ray–Object Intersections

1. **Ray and infinite cylinder.** A cylinder of radius r, aligned with the z-axis, has implicit equation $x^2 + y^2 - r^2 = 0$. For a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ with $\mathbf{o} = (-2, -2, -2)$ and $\mathbf{d} = (1, 1, 1)$, find *all* intersection parameter values t with the infinitely long cylinder of radius 2. Write the smaller value as t_0 and the larger as t_1.

   Spring 2018, Exam 1, Question 4(a)(i)

   $t_0 =$ _____ $t_1 =$ _____

   Solution: Substitute $x(t) = y(t) = t - 2$ and radius $r = 2$ into $x^2 + y^2 = 4$:

   $$(t-2)^2 + (t-2)^2 = 4 \quad \Longrightarrow \quad (t-2)^2 = 2.$$

   Thus $\boxed{t_0 = 2 - \sqrt{2}, \qquad t_1 = 2 + \sqrt{2}}$.

10 Bounding Volume Hierarchies

1. **Brute force and BVH collision testing.** Zelda is implementing 2D collision detection between an axis-aligned box and an object consisting of a union of triangles. Assume the following data types and supplied *helper functions*; you do not need to implement the helpers.

   Spring 2020, Exam 1, Question 4(e)(i)–(ii)

   ```
   struct Vector2D { float x, y; };
   struct Triangle { Vector2D v1, v2, v3; };
   struct Box { Vector2D lowerLeft, upperRight; };
   struct Object { vector<Triangle> triangles; };
   bool boxTriangleTest(const Box &b, const Triangle &tri);
   ```

   (a) Implement the brute-force intersection test with `boxTriangleTest`.

   ```
   bool boxObjectTest(const Box &b, const Object &obj) {
       // Your code here
   }
   ```

   Solution:

   ```
   bool boxObjectTest(const Box &b, const Object &obj) {
       for (const Triangle &tri : obj.triangles) {
           if (boxTriangleTest(b, tri)) return true;
       }
       return false;
   }
   ```

   Test each triangle; stop immediately on the first intersection. Otherwise return false.

(b) Now suppose the object is represented as a BVH. The pointers `left` and `right` are valid only for internal nodes; `triangles` is populated only for leaf nodes. Using the supplied `boxBoxTest` and `boxTriangleTest`, implement the recursive query.

```
struct BVHNode {
    Box boundingBox;
    bool isLeafNode;
    BVHNode *left, *right;
    vector<Triangle> triangles;
};
bool boxBoxTest(const Box &b1, const Box &b2);

bool boxObjectTestBVH(const Box &b, BVHNode *rootNode) {
    // Your code here
}
```

Solution:

```
bool boxObjectTestBVH(const Box &b, BVHNode *rootNode) {
    if (!rootNode) return false;
    if (!boxBoxTest(b, rootNode->boundingBox)) return false;
    if (rootNode->isLeafNode) {
        for (const Triangle &tri : rootNode->triangles)
            if (boxTriangleTest(b, tri)) return true;
        return false;
    }
    return boxObjectTestBVH(b, rootNode->left) ||
           boxObjectTestBVH(b, rootNode->right);
}
```

The bounding-box test prunes subtrees; leaf nodes use exact box–triangle tests. A null subtree cannot intersect the query box.